Graphic Era deemed to be University DEHRADUN

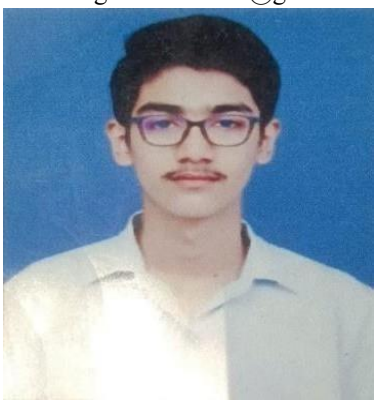
PROJECT AND TEAM INFORMATION

Project Title

Implementation of Loop Optimization Techniques

Student/Team Information

Team Name: Team ID:	OP Code CD-VI-T184
Team member 1 (Team Lead)	Faiz, Mohammad – 2023775 mohammadfaiz2337@gmail.com
Team member 2	Bari, Jass – 23021763 jassbari80@gmail.com

	Raj, Chandan – 2024018 rajchandan02468@gmail.com
Team member 3	Ramola, Vivek – 230112005 viveksinghramola1234@gmail.com 

PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

Loops are one of the most frequently executed constructs in any program and often consume a significant portion of execution time. Inefficient loops can lead to unnecessary computations, redundant operations, and poor performance, especially in applications involving large datasets, scientific computing, and real-time systems.

This project presents a **source-to-source loop optimization tool implemented in C++**, which analyzes and transforms unoptimized C/C++ code to improve performance. The system detects loop constructs (for and while) and applies multiple compiler optimization techniques such as **Loop Invariant Code Motion (LICM)**, **Strength Reduction**, **Dead Code Elimination**, and **Loop Unrolling**.

The tool reads input source code, processes it using parsing and pattern matching techniques, and generates an optimized version of the code. Additionally, it includes a performance evaluation module that compares the execution time of the original and optimized program.

Updated Project Approach and Architecture (2 pts)

The project implements a **source-to-source loop optimization engine in C++** that analyzes and transforms input C/C++ code.

The workflow is:

1. **Input Handling**
   - The system reads source code from file (input.c) using file streams
   - Implemented in main driver loop_optimizer
2. **Loop Detection Module**
   - Detects for and while loops using parsing and pattern matching
   - Implemented using custom parsing logic (detectLoops) in loop_optimizer_engine
3. **Optimization Engine**
   - **Loop Invariant Code Motion (LICM)** → hoists invariant expressions outside loop
   - **Strength Reduction** → converts multiplication into bit-shift operations
   - **Dead Code Elimination** → removes unused variables/statements
   - **Loop Unrolling** → unrolls simple loops for performance
4. **Code Generation**
   - Optimized code is written into output.c
5. **Performance Module**
   - Compiles and compares execution time of original vs optimized code

Task Completed

Task Completed	Team Member
• Implemented loop detection logic (for /While parsing) • Developed optimization engine (LICM, strength, reduction) • Implemented dead code elimination logic • Implemented loop unrolling for simple loops • Built CLI tool for input/output handling • Developed performance comparison module • Created test cases with multiple loop patterns	Faiz Chandan Jass Faiz Jass Vivek All members

Challenges/Roadblocks (7 pts)

- Parsing loops correctly without a full compiler (used regex + manual parsing)
- Handling nested loops safely without breaking logic
- Identifying true loop-invariant expressions
- Ensuring optimized code remains syntactically correct
- Avoiding incorrect dead code removal
- Handling edge cases like control statements (break, continue)
- Maintaining balance between optimization and correctness

Tasks Pending

Task Pending	Team Member (to complete the task)
• Improve nested loop optimization • Add loop fusion optimization • Enhance visualization (graph comparison UI) • Improve parser robustness • Add more edge case testing	Chandan Faiz Vivek Jass All members

Project Outcome/Deliverables (2 pts)

- A **command-line tool** that:
  - Reads C/C++ code
  - Detects loops
  - Applies multiple loop optimizations
  - Generates optimized code (output.c)
- Displays:
  - Number of loops detected
  - Number of optimizations applied
- Provides **execution time comparison**

Progress Overview (2 pts)

The project is in an **advanced stage**, where core optimization techniques such as LICM, loop unrolling, strength reduction, and dead code elimination are successfully implemented and tested on sample programs.

Codebase Information (2 pts)

- Language: C++
- Key Files:
  - loop_optimizer_engine.cpp → core optimization logic
  - loop_optimizer.cpp → main driver & CLI
  - performance.cpp → execution comparison
  - input.c → test input program t
- Libraries Used:
  - <regex> → pattern matching
  - <chrono> → performance measurement
  - STL containers (vector, unordered_set)

Testing and Validation Status (2 pts)

Test Type	Status (Pass/Fail)	Notes
Unit Testing	Pass	Individual optimizations working
Integration Testing	Pass	End-to-end flow working
Performance Testing	Pass	Execution time comparison done
Edge case Testing	In progress	Nested loops and complex cases

Deliverables Progress (2 pts)

- Code Implementation: 85% Complete
- Optimization Engine: 80% Complete
- Performance Module: 90% Complete
- Testing: 70% Complete
- Documentation: 80% Complete